Supplementary Material for Differentiable Polyphase Filtering for Large-Kernel Approximation

ZHE CAO*, State Key Lab of CAD&CG, Zhejiang University, China
ZHIZHEN WU*, State Key Lab of CAD&CG, Zhejiang University, China
ZHONGGUI CHEN, School of Informatics, Xiamen University, China
RUI WANG, State Key Lab of CAD&CG, Zhejiang University, China
YUCHI HUO†, State Key Lab of CAD&CG, Zhejiang University, China

1 Reduced Dimension Parameter Space

In contrast to prior methods that optimize an independent offset and weight for every sample, we adopt a geometry-aware grouped parameterization that substantially reduces the search space. We partition samples according to the kernel geometry and tie parameters within each group: all samples in a group share a single weight, while their offsets are generated from a common dilation factor. Concretely, each sample uses a fixed base offset $\mathbf{o}_{\text{base}}$, and the effective offset is $\mathbf{o} = \gamma, \mathbf{o}_{\text{base}}$. This low-dimensional formulation restricts the degrees of freedom relative to fully free kernels, making optimization more stable and less prone to poor local minima. We initialize from Dual Kawase topologies [Martin et al. 2015] and optimize shared parameters layer-wise.

For downsampling, we use a center sample and a symmetric diagonal group, optimizing only γ and the center weight ω_c, with diagonal weights given by $\omega_d = (1 - \omega_c)/4$. For upsampling, we use diagonal and axial groups with separate dilations (γ_d, γ_a), and enforce the partition of unity by optimizing one weight (e.g., ω_d) and setting $\omega_a = (1 - 4\omega_d)/4$.

2 Optimal-level assignment for a parameter map

We precompute a kernel-size–to–level LUT by comparing adjacent levels $(L, L+1)$ with Polyphase Filtering and selecting the smallest level that satisfies $\mathcal{L}_L \le \mathcal{L}_{L+1}$; when the inequality flips, we advance the baseline and continue with $(L+1, L+2)$. Algorithm 1 summarizes this crossover-based procedure and the resulting LUT construction.

Algorithm 1: Crossover-based LUT construction for layer-adaptive filtering.

struct *LayerAdaptiveLUT*
  function *BuildLUT*($\mathcal{K}$, $L_{\max}$)
    for $k \in \mathcal{K}$ **do**
      LUT[k] $\leftarrow 0$;
    $k_{\min} \leftarrow \min(\mathcal{K})$;
    for $L \leftarrow 0$ **to** $L_{\max} - 1$ **do**
      for $k \in \mathcal{K}$ **do**
        $\mathcal{L}_L(k) \leftarrow \text{PolyphaseFiltering}(k, L)$;
        $\mathcal{L}_{L+1}(k) \leftarrow \text{PolyphaseFiltering}(k, L{+}1)$;
      `// First crossover where` $\mathcal{L}_L(k) > \mathcal{L}_{L+1}(k)$
      $k^\star \leftarrow \min\{k \in \mathcal{K} \mid k \ge k_{\min} \ \wedge \ \mathcal{L}_L(k) > \mathcal{L}_{L+1}(k)\}$;
      if $k^\star$ *is undefined* **then** `// no crossover at this baseline`
        $k^\star \leftarrow \max(\mathcal{K})$;
      for $k \in \mathcal{K}$ **s.t.** $k_{\min} \le k \le k^\star$ **do**
        LUT[k] $\leftarrow L$;
      $k_{\min} \leftarrow \min\{k \in \mathcal{K} \mid k > k^\star\}$;
      if $k_{\min}$ *is undefined* **then**
        return LUT;
    return LUT;

Input: Discrete target kernel sizes $\mathcal{K}$, maximum level $L_{\max}$.
Output: Kernel-size-to-level LUT.

LUT $\leftarrow$ BuildLUT($\mathcal{K}, L_{\max}$);
return LUT;

3 Additional Performance Analysis

For the configurations in Table 1, we analyze the runtime performance on an NVIDIA GeForce RTX 3090 GPU across varying resolutions and geometric kernel sizes. The results highlight two critical scalability advantages of our framework. First, kernel size scalability: increasing the kernel diameter (and consequently adding an extra hierarchical level) incurs negligible latency overhead for our method. For instance, at 4K resolution, the transition from a 2-level to a 3-level hierarchy increases inference time by only ~ 0.01 ms. Second, resolution scalability: as the resolution scales from 1080p to 4K, our method exhibits a significantly lower latency increase compared to the baselines. This indicates that our multi-resolution strategy effectively alleviates memory bandwidth bottlenecks. Consequently, the relative efficiency gap widens as the computational load increases.

4 Complex Kernel

For the letter i bokeh, we compare our method (configured with a 2-level hierarchy (2 downsampling and 2 upsampling stages), with 8, 8, 18, 18 samples for each layer, respectively) against the optimization-based baselines, PST and DSKC (4 passes with 10 samples each). For

*Equal contribution
†Corresponding author

Authors' Contact Information: Zhe Cao, caozhe022@qq.com, State Key Lab of CAD&CG, Zhejiang University, China; Zhizhen Wu, zhizhenwu@zju.edu.cn, State Key Lab of CAD&CG, Zhejiang University, China; Zhonggui Chen, chenzhonggui@xmu.edu.cn, School of Informatics, Xiamen University, China; Rui Wang, ruiwang@zju.edu.cn, State Key Lab of CAD&CG, Zhejiang University, China; Yuchi Huo, huo.yuchi.sc@gmail.com, State Key Lab of CAD&CG, Zhejiang University, China.

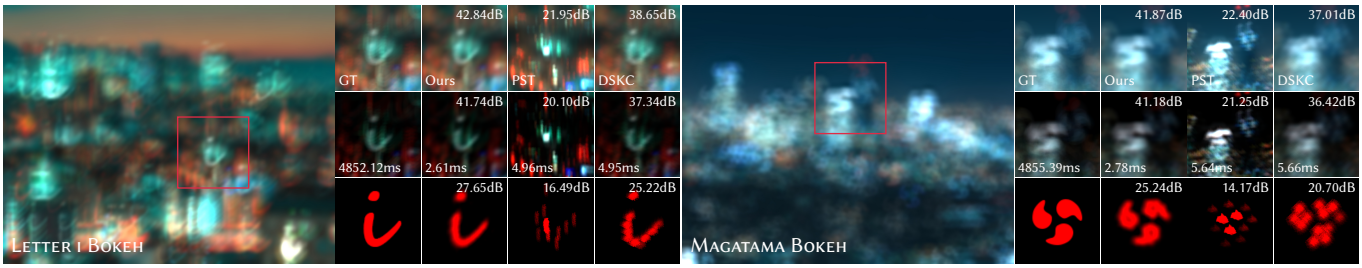

Fig. 1. **Visual comparison of bokeh rendering with complex kernels**. We compare our method against optimization-based sparse baselines (PST, DSKC). The rows display, from top to bottom: **(1)** Final composited bokeh results; **(2)** Filtered highlights annotated with runtime latency (in ms); **(3)** Kernel approximation visualizations, labeled with PSNR to measure kernel structure and details.

Table 1. **Runtime comparison (in milliseconds) across different resolutions and kernel sizes.** Dense (GT) denotes the exact spatial convolution baseline. Our method achieves sub-millisecond latency even at 4K resolution (< 0.21ms) and offers an over 2.6× speedup compared to sparse baselines (PST, DSKC).

Kernel Size	**109 × 109**			**219 × 219**		
Resolution	**1080p**	**2K**	**4K**	**1080p**	**2K**	**4K**
GT	121.62	219.44	505.22	487.87	888.15	2031.12
Ours	**0.0802**	**0.1081**	**0.1911**	**0.0927**	**0.1221**	**0.2063**
PST	0.1519	0.2109	0.4506	0.1627	0.2525	0.5381
DSKC	0.1525	0.2098	0.4500	0.1631	0.2530	0.5375
SpeedUp	1.8948	1.9459	2.3564	1.7573	2.0700	2.6069

the magatama bokeh, we compare our method (configured with a 2-level hierarchy (2 downsampling and 2 upsampling stages), with 8, 8, 24, 24 samples for each layer, respectively) against the optimization-based baselines, PST and DSKC (4 passes with 12 samples each). As demonstrated in Fig. 1, our method can approximate disconnected or highly asymmetric kernels, such as characters, logos, and symbols.

5 Quantitative Results of Compared Methods

We divide the experiments presented in the main text into two tables (Table 2 and Table 3). One table includes results for the Gaussian filter, covering the single Gaussian kernel experiment (Bloom) and the spatially varying Gaussian blur experiment (Scratched Liquid Glass). The other table presents results for Geometric kernels, including the single geometric kernel experiment and the spatially varying geometric kernel experiment. We evaluate these experiments using three quality metrics (PSNR, SSIM, LPIPS) and two performance metrics (Mobile time and PC time). Compared to PST [Schuster et al. 2020] and DSKC [Wu et al. 2026], our method achieves nearly the best results across all experiments and metrics, demonstrating its advanced and general applicability.

6 Optimization Process Analysis

As shown in Fig. 2, we evaluate the optimization dynamics of the Gaussian kernel against the Side4 kernel. While the Side4 kernel converges more rapidly, it plateaus at a significantly higher MSE loss compared to the Gaussian kernel. This confirms that geometric kernels are inherently more challenging to approximate than

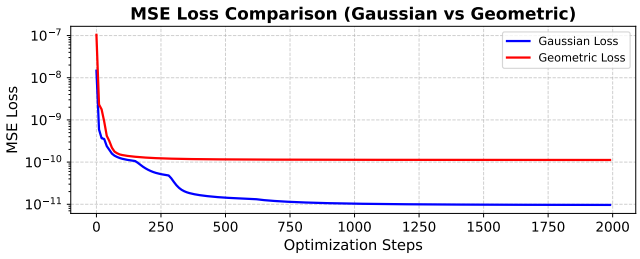

Fig. 2. **Optimization dynamics of Gaussian vs. geometric kernels.** Comparison of the MSE convergence curves when approximating smooth Gaussian and sharp geometric (Side4) kernels using our method.

smooth ones. Theoretically, this disparity is explained by the Central Limit Theorem for convolutions: iterated convolution naturally tends toward a Gaussian distribution [Lindeberg 2013], rendering sharp, non-Gaussian structures more difficult to preserve during optimization. As a result, the Gaussian kernel can be more easily approximated through multi-pass filtering.

References

Tony Lindeberg. 2013. *Scale-space theory in computer vision*. Vol. 256. Springer Science & Business Media.

Sam Martin, Andrew Garrard, Andrew Gruber, Marius Bjorge, Renaldas Zioma, Simon Benge, and Niklas Nummelin. 2015. Moving mobile graphics. In *ACM SIGGRAPH 2015 Courses* (Los Angeles, California) *(SIGGRAPH '15)*. Association for Computing Machinery, New York, NY, USA, Article 18. doi:10.1145/2776880.2787664

Kersten Schuster, Philip Trettner, and Leif Kobbelt. 2020. High-performance image filters via sparse approximations. *Proceedings of the ACM on Computer Graphics and Interactive Techniques* 3, 2 (2020), 1–19.

Zhizhen Wu, Zhe Cao, and Yuchi Huo. 2026. DISK: Differentiable Sparse Kernel Complex for Efficient Spatially-Variant Convolution. In *The Fourteenth International Conference on Learning Representations*. https://openreview.net/forum?id=bbuxDoRD2D

Table 2. **Gaussian kernels results**. A comprehensive evaluation of the single Gaussian kernel experiment (Bloom) and the spatially varying Gaussian blur experiment (Scratched Liquid Glass) presented in the main text is conducted using three quality metrics and two performance metrics.

Effect	Metric	GT	Ours	Multi-Box	Kawase	Dual	PST	DSKC
Bloom	PSNR ↑	—	**67.77**	46.38	27.92	46.56	33.79	26.89
	SSIM ↑	—	**0.9999**	0.9994	0.9577	0.9987	0.9870	0.9479
	LPIPS ↓	—	**2.27e-5**	0.0007	0.0479	0.0008	0.0108	0.0456
	PC (ms) ↓	2.27	**0.0946**	1.6950	0.1631	0.1275	0.1630	0.1631
	Mob. (ms) ↓	27.67	**2.21**	14.70	5.27	2.56	5.29	5.27
Liquid Glass	PSNR ↑	—	**54.15**	—	49.45	47.30	39.22	50.90
	SSIM ↑	—	**0.9992**	—	0.9983	0.9969	0.9872	0.9988
	LPIPS ↓	—	**0.0019**	—	0.0040	0.0064	0.0214	0.0030
	PC (ms) ↓	29.33	**0.0663**	—	0.1438	0.0911	0.1433	0.1435
	Mob. (ms) ↓	223.57	**0.94**	—	2.55	1.26	2.56	2.58

Table 3. **Geometric kernels results**. A comprehensive evaluation of the single geometric kernel experiment and the spatially varying geometric kernel experiment presented in the main text is conducted using three quality metrics and two performance metrics.

		Single Kernel						Spatially Varying Kernel					
Metric	**Method**	**Circle**	**Star4**	**Ring**	**Side4**	**Lett. i**	**Maga.**	**Circle**	**Star4**	**Ring**	**Side4**	**Rotat.**	**Radial**
PSNR ↑	Ours	**50.85**	**48.85**	**44.44**	**53.06**	**43.01**	**49.08**	**36.77**	**39.04**	**35.59**	**39.70**	**40.51**	**42.63**
	PST	38.19	37.33	34.60	40.12	21.96	28.96	24.48	29.00	24.47	25.61	25.83	30.26
	DSKC	34.79	43.30	32.60	42.46	38.71	44.35	32.39	34.44	29.06	35.59	28.25	33.20
SSIM ↑	Ours	**0.9976**	**0.9961**	**0.9928**	**0.9980**	**0.9833**	**0.9943**	**0.9627**	**0.9652**	**0.9434**	**0.9797**	**0.9937**	**0.9937**
	PST	0.9831	0.9774	0.9550	0.9655	0.6670	0.8189	0.8423	0.8759	0.7971	0.8756	0.8957	0.9531
	DSKC	0.9614	0.9899	0.9465	0.9757	0.9713	0.9882	0.9428	0.9556	0.8853	0.9739	0.9582	0.9782
LPIPS ↓	Ours	**0.0190**	**0.0237**	**0.0430**	**0.0294**	**0.1641**	0.1618	**0.1387**	**0.1356**	**0.1375**	**0.0880**	**0.0509**	**0.0122**
	PST	0.0983	0.0800	0.3201	0.2284	0.4526	0.3334	0.2425	0.2238	0.3230	0.2274	0.4031	0.1099
	DSKC	0.1325	0.0565	0.2908	0.2299	0.1864	**0.1585**	0.2038	0.1517	0.2742	0.0981	0.2024	0.0452
PC (ms) ↓	GT	121.03	121.42	120.60	121.65	487.85	487.90	123.94	122.62	122.82	121.62	591.79	749.85
	Ours	**0.0820**	**0.0820**	**0.0810**	**0.0830**	**0.0982**	**0.1073**	**0.0841**	**0.0840**	**0.0840**	**0.0842**	**0.0841**	**0.1651**
	PST	0.1361	0.1365	0.1366	0.1362	0.2195	0.2556	0.1828	0.1830	0.1831	0.1828	0.1828	0.3656
	DSKC	0.1360	0.1362	0.1362	0.1365	0.2193	0.2557	0.1829	0.1828	0.1829	0.1830	0.1829	0.3654
Mob. (ms) ↓	GT	1214.2	1223.3	1212.7	1205.3	4982.7	4951.3	1266.3	1262.1	1260.5	1271.9	6218.7	8706.2
	Ours	**2.12**	**2.12**	**2.12**	**2.11**	**2.61**	**2.78**	**2.15**	**2.14**	**2.16**	**2.15**	**2.15**	**4.31**
	PST	4.39	4.39	4.38	4.37	4.96	5.64	5.30	5.35	5.31	5.31	4.29	8.52
	DSKC	4.42	4.39	4.40	4.38	4.95	5.66	5.35	5.33	5.29	5.36	4.25	8.56